

Análisis y desarrollo de software

Aprendiz: yira Isabel Contreras Espinosa

Instructor: GUSTAVO ADOLFO

FICHA:3134547

**Evidencia: Diseño y Desarrollo de servicios web –
proyecto GA7-220501096-AA5-EV03**

```
<!DOCTYPE html>

<html lang="es">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>GA7-220501096-AA5-EV03 - Diseño y Desarrollo de Servicios Web</title>

  <link
href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;600;700&family=Inter:wght@400;500;700&display=swap" rel="stylesheet">

  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.1/css/all.min.css">

  <style>

    :root {

      --primary: #064e3b; /* Verde SENA */

      --primary-light: #ecfdf5;

      --accent: #10b981;

      --dark: #0f172a;

      --light: #f8fafc;
```

```
--border: #e2e8f0;
--text-muted: #64748b;
}
```

```
*{
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}
```

```
body {
  font-family: 'Inter', sans-serif;
  background-color: #f1f5f9;
  color: #334155;
  line-height: 1.6;
  padding: 40px 20px;
}
```

```
.container {
  max-width: 1000px;
  margin: 0 auto;
  background-color: #ffffff;
  border-radius: 16px;
  box-shadow: 0 4px 20px rgba(0,0,0,0.05);
  overflow: hidden;
  border: 1px solid var(--border);
}
```

```
}
```

```
/* Encabezado */
```

```
header {  
    background-color: var(--primary);  
    color: #ffffff;  
    padding: 50px;  
    position: relative;  
}
```

```
header h1 {  
    font-family: 'Poppins', sans-serif;  
    font-size: 32px;  
    margin-bottom: 10px;  
    font-weight: 700;  
}
```

```
header h2 {  
    font-size: 18px;  
    font-weight: 400;  
    color: #a7f3d0;  
    margin-bottom: 30px;  
}
```

```
.meta-grid {  
    display: grid;
```

```
    grid-template-columns: repeat(2, 1fr);  
    gap: 20px;  
    border-top: 1px solid rgba(255,255,255,0.1);  
    padding-top: 25px;  
}
```

```
.meta-item span.label {  
    display: block;  
    font-size: 11px;  
    text-transform: uppercase;  
    letter-spacing: 1px;  
    color: #a7f3d0;  
    font-weight: 600;  
}
```

```
.meta-item span.val {  
    font-size: 15px;  
    font-weight: 500;  
}
```

```
/* Contenido */  
.content {  
    padding: 50px;  
}
```

```
section {
```

```
margin-bottom: 45px;
}
```

```
h2.section-title {
  font-family: 'Poppins', sans-serif;
  font-size: 22px;
  color: var(--primary);
  margin-bottom: 20px;
  display: flex;
  align-items: center;
  gap: 10px;
  border-bottom: 2px solid var(--border);
  padding-bottom: 8px;
}
```

```
p {
  margin-bottom: 15px;
  color: #475569;
}
```

```
ul {
  list-style: none;
  margin-bottom: 20px;
}
```

```
ul li {
```

```
position: relative;

padding-left: 25px;

margin-bottom: 10px;
}
```

```
ul li::before {

    content: "\f00c";

    font-family: "Font Awesome 5 Free";

    font-weight: 900;

    position: absolute;

    left: 0;

    color: var(--accent);

}
```

```
/* Tablas */

table {

    width: 100%;

    border-collapse: collapse;

    margin: 20px 0;

}
```

```
th, td {

    padding: 12px 15px;

    border-bottom: 1px solid var(--border);

    text-align: left;

    font-size: 14px;
```

```
}
```

```
th {  
    background-color: var(--primary-light);  
    color: var(--primary);  
    font-weight: 600;  
}
```

```
/* Insignias de Verbos HTTP */
```

```
.badge {  
    padding: 4px 8px;  
    border-radius: 4px;  
    font-weight: 700;  
    font-size: 12px;  
    display: inline-block;  
}
```

```
.badge.get { background-color: #dbeafe; color: #1e40af; }  
.badge.post { background-color: #dcfd5; color: #065f46; }  
.badge.put { background-color: #fef3c7; color: #92400e; }  
.badge.delete { background-color: #fee2e2; color: #991b1b; }
```

```
/* Bloques de Código */
```

```
pre {  
    background-color: var(--dark);  
    color: #f8fafc;
```

```
padding: 20px;
border-radius: 8px;
overflow-x: auto;
font-family: 'Courier New', Courier, monospace;
font-size: 13px;
margin: 15px 0;
border-left: 4px solid var(--accent);
}
```

```
code {
  font-family: inherit;
}
```

```
.info-box {
  background-color: #f0fdf4;
  border: 1px solid #bbf7d0;
  padding: 20px;
  border-radius: 8px;
  color: #166534;
  margin: 20px 0;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="container">
```

<header>

<h1>INFORME TÉCNICO: SERVICIOS WEB REST</h1>

<h2>Evidencia: GA7-220501096-AA5-EV03 - Diseño y desarrollo de servicios web
- Proyecto</h2>

<div class="meta-grid">

<div class="meta-item">

Proyecto Formativo

SELECTO - Gestión de Candidatos y Selección

</div>

<div class="meta-item">

Ficha de Formación

3134547

</div>

<div class="meta-item">

Instructor Líder

Gustavo Adolfo

</div>

<div class="meta-item">

Centro de Formación

Centro de Comercio y Servicios

</div>

</div>

</header>

<div class="content">

<!-- SECCIÓN 1: INTRODUCCIÓN -->

<section>

<h2 class="section-title"><i class="fa-solid fa-circle-info"></i> 1.
Introducción</h2>

<p>

En el desarrollo moderno de software, la interoperabilidad y la modularidad de los sistemas se logran mediante el uso de servicios web. Los servicios web de tipo REST (Representational State Transfer) permiten el intercambio fluido de datos estructurados (generalmente en formato JSON) entre sistemas heterogéneos mediante el protocolo HTTP estándar.

</p>

<p>

Este documento técnico documenta el diseño e implementación de la API de servicios web para el sistema de información Selecto. El objetivo de este servicio es centralizar las operaciones CRUD básicas sobre los candidatos registrados, abriendo la puerta a futuras integraciones con terminales móviles u otras plataformas de manera desacoplada.

</p>

</section>

<!-- SECCIÓN 2: DISEÑO DE LA API -->

<section>

<h2 class="section-title"><i class="fa-solid fa-gears"></i> 2. Diseño de los
Endpoints</h2>

<p>

Para el proyecto formativo, se ha diseñado un controlador de servicios bajo la ruta base <code>/api/candidatos</code>. A continuación se detallan los endpoints REST disponibles para interactuar con la tabla de datos:

</p>

```

<table>

  <thead>

    <tr>

      <th>Método</th>

      <th>Endpoint</th>

      <th>Descripción</th>

      <th>Formato de Entrada</th>

      <th>Respuesta Esperada</th>

    </tr>

  </thead>

  <tbody>

    <tr>

      <td><span class="badge get">GET</span></td>

      <td><code>/api/candidatos</code></td>

      <td>Obtener lista de todos los candidatos</td>

      <td>Ninguno</td>

      <td>JSON (Array de Candidatos)</td>

    </tr>

    <tr>

      <td><span class="badge post">POST</span></td>

      <td><code>/api/candidatos</code></td>

      <td>Registrar un nuevo candidato</td>

      <td>JSON (Body Payload)</td>

      <td>JSON (Estado de Éxito / Error)</td>

    </tr>

  </tbody>

</table>

```

<td>PUT</td>
<td><code>/api/candidatos</code></td>
<td>Actualizar datos de un candidato</td>
<td>JSON (Body con id e información)</td>
<td>JSON (Estado de Éxito / Error)</td>
</tr>
<tr>
<td>DELETE</td>
<td><code>/api/candidatos?id={id}</code></td>
<td>Eliminar físicamente un candidato</td>
<td>Parámetro de consulta (Query param)</td>
<td>JSON (Estado de Éxito / Error)</td>
</tr>
</tbody>
</table>
</section>

<!-- SECCIÓN 3: CÓDIGO FUENTE IMPLEMENTADO -->

<section>

<h2 class="section-title"><i class="fa-solid fa-code"></i> 3. Código Fuente del Servicio Web (Java Servlet API)</h2>

<p>

Dado que el proyecto formativo se desarrolla utilizando la pila Java Web (JEE) con Servlets y Maven, se implementó el siguiente controlador controlador REST llamado <code>CandidatoApiController.java</code>, el cual analiza el método HTTP entrante para realizar la correspondiente consulta SQL utilizando nuestra conexión JDBC (<code>ConexionBD</code>).

</p>

```

<pre><code>package org.example;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.BufferedReader;
import java.io.IOException;
import java.io.PrintWriter;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

@WebServlet("/api/candidatos")
public class CandidatoAPIController extends HttpServlet {

    // 1. GET: Retorna la lista de candidatos en formato estructurado JSON

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws
ServletException, IOException {

        resp.setContentType("application/json;charset=UTF-8");

        PrintWriter out = resp.getWriter();

        StringBuilder jsonResponse = new StringBuilder();

```

```

jsonResponse.append("[");

try {

    Connection con = ConexionBD.getConexion();

    String sql = "SELECT id, nombre, correo FROM candidatos";

    PreparedStatement ps = con.prepareStatement(sql);

    ResultSet rs = ps.executeQuery();

    boolean primerRegistro = true;

    while (rs.next()) {

        if (!primerRegistro) {

            jsonResponse.append(",");

        }

        jsonResponse.append("{")

            .append("\"id\":").append(rs.getInt("id")).append(",")

            .append("\"nombre\":\").append(rs.getString("nombre")).append("\",")

            .append("\"correo\":\").append(rs.getString("correo")).append("\",")

            .append("}");

        primerRegistro = false;

    }

    rs.close();

    ps.close();

    con.close();

} catch (Exception e) {

    resp.setStatus(HttpServletResponse.SC_INTERNAL_SERVER_ERROR);

    out.print("{\"status\": \"error\", \"message\": \"\" + e.getMessage() + \"\"}");

    0;

```

```
}  
    jsonResponse.append("]");  
    out.print(jsonResponse.toString());  
}
```

// 2. POST: Recibe datos para insertar un candidato en la base de datos

@Override

protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws
ServletException, IOException {

resp.setContentType("application/json;charset=UTF-8");

PrintWriter out = resp.getWriter();

// Leer el body enviado en formato JSON

String body = readRequestBody(req);

// Parsing manual sencillo de JSON

String nombre = getJsonValue(body, "nombre");

String correo = getJsonValue(body, "correo");

if (nombre.isEmpty() || correo.isEmpty()) {

resp.setStatus(HttpServletResponse.SC_BAD_REQUEST);

out.print("{\"status\":\"error\",\"message\":\"Campos obligatorios faltantes.\"}");

return;

}

try {

```

Connection con = ConexionBD.getConexion();

String sql = "INSERT INTO candidatos (nombre, correo) VALUES (?, ?)";

PreparedStatement ps = con.prepareStatement(sql);

ps.setString(1, nombre);

ps.setString(2, correo);


int rows = ps.executeUpdate();

ps.close();

con.close();


if (rows > 0) {

    resp.setStatus(HttpServletResponse.SC_CREATED);

    out.print("{\"status\":\"success\",\"message\":\"Candidato registrado correctamente.\"}");

} else {

    resp.setStatus(HttpServletResponse.SC_INTERNAL_SERVER_ERROR);

    out.print("{\"status\":\"error\",\"message\":\"No se pudo registrar.\"}");

}

} catch (Exception e) {

    resp.setStatus(HttpServletResponse.SC_INTERNAL_SERVER_ERROR);

    out.print("{\"status\":\"error\",\"message\":\"\" + e.getMessage() + "\"}");

}

}

```

// 3. PUT: Actualiza la información de un candidato existente

@Override

```
protected void doPut(HttpServletRequest req, HttpServletResponse resp) throws  
ServletException, IOException {
```

```
    resp.setContentType("application/json;charset=UTF-8");
```

```
    PrintWriter out = resp.getWriter();
```

```
    String body = readRequestBody(req);
```

```
    String idStr = getJsonValue(body, "id");
```

```
    String nombre = getJsonValue(body, "nombre");
```

```
    String correo = getJsonValue(body, "correo");
```

```
    if (idStr.isEmpty() || nombre.isEmpty() || correo.isEmpty()) {
```

```
        resp.setStatus(HttpServletResponse.SC_BAD_REQUEST);
```

```
        out.print("{\"status\":\"error\",\"message\":\"Campos obligatorios  
incompletos.\"}");
```

```
        return;
```

```
    }
```

```
    try {
```

```
        Connection con = ConexionBD.getConexion();
```

```
        String sql = "UPDATE candidatos SET nombre = ?, correo = ? WHERE id = ?";
```

```
        PreparedStatement ps = con.prepareStatement(sql);
```

```
        ps.setString(1, nombre);
```

```
        ps.setString(2, correo);
```

```
        ps.setInt(3, Integer.parseInt(idStr));
```

```
        int rows = ps.executeUpdate();
```

```
        ps.close();
```

```

        con.close();

        if (rows > 0) {

            out.print("{\"status\":\"success\",\"message\":\"Candidato actualizado satisfactoriamente.\"}");

        } else {

            resp.setStatus(HttpServletResponse.SC_NOT_FOUND);

            out.print("{\"status\":\"error\",\"message\":\"No se encontró el id especificado.\"}");

        }

    } catch (Exception e) {

        resp.setStatus(HttpServletResponse.SC_INTERNAL_SERVER_ERROR);

        out.print("{\"status\":\"error\",\"message\":\"\" + e.getMessage() + "\"}");

    }

}

```

// 4. DELETE: Elimina un registro utilizando un parámetro de consulta

```

@Override

protected void doDelete(HttpServletRequest req, HttpServletResponse resp)
throws ServletException, IOException {

    resp.setContentType("application/json;charset=UTF-8");

    PrintWriter out = resp.getWriter();

    String idStr = req.getParameter("id");

    if (idStr == null || idStr.isEmpty()) {

        resp.setStatus(HttpServletResponse.SC_BAD_REQUEST);

        out.print("{\"status\":\"error\",\"message\":\"Falta el ID del candidato.\"}");
    }
}

```

```

        return;
    }

    try {
        Connection con = ConexionBD.getConexion();

        String sql = "DELETE FROM candidatos WHERE id = ?";

        PreparedStatement ps = con.prepareStatement(sql);

        ps.setInt(1, Integer.parseInt(idStr));

        int rows = ps.executeUpdate();

        ps.close();

        con.close();

        if (rows > 0) {
            out.print("{\"status\":\"success\",\"message\":\"Candidato eliminado satisfactoriamente.\"}");
        } else {
            resp.setStatus(HttpServletResponse.SC_NOT_FOUND);

            out.print("{\"status\":\"error\",\"message\":\"El id especificado no existe.\"}");
        }
    } catch (Exception e) {
        resp.setStatus(HttpServletResponse.SC_INTERNAL_SERVER_ERROR);

        out.print("{\"status\":\"error\",\"message\":\"\" + e.getMessage() + "\"}");
    }
}

```

```
// Función auxiliar para leer los cuerpos JSON de las solicitudes (POST/PUT)
private String readRequestBody(HttpServletRequest req) throws IOException {
    StringBuilder sb = new StringBuilder();
    String line;
    try (BufferedReader reader = req.getReader()) {
        while ((line = reader.readLine()) != null) {
            sb.append(line);
        }
    }
    return sb.toString();
}
```

```
// Parsing primitivo para extraer llaves/valores de JSON sin dependencias externas
private String getJsonValue(String json, String key) {
    String pattern = "\"" + key + "\"\":";
    int index = json.indexOf(pattern);
    if (index != -1) {
        int start = index + pattern.length();
        int end = json.indexOf("\"", start);
        return json.substring(start, end);
    }
    // Si es número
    pattern = "\"" + key + "\":";
    index = json.indexOf(pattern);
    if (index != -1) {
        int start = index + pattern.length();
```

```
int end = json.indexOf(";", start);  
if (end == -1) end = json.indexOf("}", start);  
return json.substring(start, end).trim();  
}  
return "";  
}
```

}</code></pre>

</section>

<!-- SECCIÓN 4: MODELO DE RESPUESTAS JSON -->

<section>

<h2 class="section-title"><i class="fa-solid fa-circle-check"></i> 4. Modelos de Respuestas (Ejemplos JSON)</h2>

<p>

Los payloads de respuesta están diseñados para ser claros y fáciles de analizar por cualquier cliente frontend (React, Angular o móvil):

</p>

<h3>Respuesta Exitosa de Consulta (GET)</h3>

<pre><code>[

{

"id": 1,

"nombre": "Yira Contreras",

"correo": "yira.contreras@example.com"

},

{

"id": 2,

```
"nombre": "Gustavo Adolfo",  
"correo": "gustavo.adolfo@sena.edu.co"  
}  
]</code></pre>
```

<h3>Respuesta de Operación Exitosa (POST, PUT, DELETE)</h3>

```
<pre><code>{  
"status": "success",  
"message": "Candidato registrado correctamente."  
}</code></pre>
```

<h3>Respuesta en Caso de Error (Ejemplo 400 Bad Request)</h3>

```
<pre><code>{  
"status": "error",  
"message": "Campos obligatorios incompletos."  
}</code></pre>
```

</section>

<!-- SECCIÓN 5: PRUEBAS EN POSTMAN -->

<section>

<h2 class="section-title"><i class="fa-solid fa-vial"></i> 5. Preparación para Pruebas (Postman)</h2>

<div class="info-box">

¿Qué sigue en la evidencia GA7-220501096-AA5-EV04?

Esta implementación cumple con el diseño y desarrollo del código. El siguiente paso técnico es descargar la herramienta Postman para

simular las peticiones HTTP y capturar las respuestas JSON que validan el correcto funcionamiento del software.

</div>

<p>Pasos sugeridos para verificar la API:</p>

Probar GET: Envía una petición <code>GET</code> a la URL <code>http://localhost:8080/seleccion/api/candidatos</code>.

Probar POST: Cambia el verbo a <code>POST</code>, ve a la pestaña Body, selecciona raw y en formato cambia a JSON. Agrega un cuerpo de prueba.

Probar DELETE: Envía un <code>DELETE</code> agregando el parámetro en la URL: <code>?id=1</code>.

</section>

<section>

<h2 class="section-title"><i class="fa-solid fa-list-check"></i> 6.
Conclusiones</h2>

<p>

La implementación de esta API REST utilizando Java Servlets nativos dentro del ecosistema Maven del proyecto Selecto demuestra una arquitectura robusta, ligera y escalable. Al no depender de pesadas librerías externas para la generación y el mapeo de formatos JSON, el sistema conserva un rendimiento de carga óptimo, garantizando una comunicación transparente con la base de datos MySQL local.

</p>

</section>

</div>

</div>

</body>

</html>b